Exercises for the report

Marco Cè

Sunday 11th October, 2026

Report and submission

The exercises below are carried out during the laboratory sessions. Each student collects the results in an individual report, written in their own words and based on their own implementations and measurements. Please send the report to the teacher as a single PDF file at least two weeks before the scheduled date of your exam. The report is discussed during the oral examination.

Every exercise follows the same layout: the subsections describe the tasks, together with the validation and measurements that go with them, and the final *Report checklist* lists the points the report should cover. Parts marked *Optional* are not expected, but they are welcome in the report if you complete them.

What to include in the report

For each exercise, describe the implementation and the procedure used to validate it. In the report, include only short code excerpts that support the discussion, together with the commands needed to build and run the program. Provide the full source code separately, for example in a GitHub or GitLab repository or a Google Drive folder. If you use a hosting service, keep the repository or folder private and share it only with the teacher. State all parameters that affect the result, such as the problem size, numerical precision, number of iterations, or dataset and model configuration.

For performance measurements, record the machine used. On WILSON, this means the partition, number and type of nodes, MPI ranks, OpenMP threads, CPU allocation, and loaded software modules. On another computer, such as your laptop, give the processor model, operating system, and compiler version. Report elapsed times together with the corresponding problem and resource sizes. Short runs should be repeated when their variability is comparable with the differences being studied.

For a strong-scaling study, keep the global problem size fixed and compute the speedup and parallel efficiency as

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p}, \tag{1}$$

where T_1 is the time of the baseline and T_p is the time with p equivalent processing elements. For weak scaling, keep the local work per processing element approximately constant, state precisely how the global problem grows, and define the efficiency convention used in the report. Present the results in plots or tables and discuss the main trends rather than reporting performance numbers in isolation.

1. The Basel problem

In 1650 the Italian mathematician Pietro Mengoli was the first to pose the problem of whether the sum of inverse squares

$$\zeta(2) = \sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \dots \tag{2}$$

can be expressed in closed form. The problem was solved by Leonhard Euler from Basel in 1734. The following year he announced that this series is exactly equal to

$$\zeta(2) = \frac{\pi^2}{6} = 1.64493406684822643647241516664602518921894990120 6\dots \tag{3}$$

This exercise puts into practice the floating-point concepts of Computer arithmetic, in particular machine epsilon and the effect of the order of operations on rounding.

Partial sums in single precision

Here, we want to verify numerically that Euler's solution is correct. Since one cannot sum infinitely many terms, let us define the partial sum

$$S(N) = \sum_{n=1}^{N} \frac{1}{n^2}. \tag{4}$$

Check numerically that, for increasing N, the partial sum $S(N)$ tends to $\pi^2/6$. Write a code that computes $S(N)$ with single-precision floating-point numbers, aiming for about seven correct decimal digits, roughly the precision of the binary32 format. Your code should:

- let you choose the N, e.g. by passing N as a command-line argument;
- perform the sum either in direct order or in reverse order;
- print N, the computed partial sum, and its absolute error with respect to $\pi^2/6$ in a machine-readable format, computing the error in double precision.

How close can you get to the exact value with single-precision arithmetic? Do the direct and reverse sums produce the same result?

Convergence study

Study the accuracy of the approximation as a function of N, up to about 10^8–10^9. What is the best way to sample N? Plot the absolute error as a function of N for both summation orders and discuss the observed behaviour. State the compiler options or Python and NumPy types used, so that the numerical precision of the calculation is unambiguous.

Some questions to guide the discussion:

- The truncation error of the partial sum is $\pi^2/6 - S(N) \approx 1/N$. In which range of N does the truncation error dominate, and where does rounding take over?
- At which N does each summation order stop improving? Can you explain this value using machine epsilon?

If you use Python, note that a pure-Python loop up to $N = 10^9$ takes tens of minutes, and an array of 10^9 single-precision values needs 4 GB of memory. Process the terms in chunks, or limit the largest N.

Optional: SIMD intrinsics

Single instructuion multiple data

Write a version of the code using SIMD intrinsics, and compare it with the scalar version in both speed and accuracy. State the compiler options of the scalar version as well: with options that allow the compiler to reorder the sum, such as `-ffast-math`, it can vectorize the scalar loop by itself. SIMD instructions and intrinsics depend on the processor architecture: for example, AVX on x86 processors and NEON on ARM processors, including Apple silicon. Check which instruction sets your machine supports, for example with `lscpu` on Linux, and state the processor model in the report. If you use WILSON, note that the `lab4` partition includes nodes of several processor generations: run the timings on a compute node through a batch job, as in Exercise 2, and select the node type with the Slurm option `--constraint`. References: https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html (x86), https://developer.arm.com/architectures/instruction-sets/intrinsics/ (ARM).

Report checklist

- the precision used: compiler options, or Python and NumPy types;
- how close the single-precision sum gets to $\pi^2/6$, and whether the direct and reverse sums agree;
- the sampling of N and a plot of the absolute error as a function of N for both summation orders;
- a discussion of the observed behaviour: where truncation and rounding errors dominate, and where each summation order stops improving.

2. Parallel Hello, World!

In this exercise you write a small program that reports how a parallel run is organized: which processes and threads are started and on which nodes they run. Besides introducing OpenMP, MPI, and Slurm, the program is a useful diagnostic tool for the later exercises: running it with the same Slurm request and environment as a real calculation shows whether the processes and threads are laid out as intended.

Shared memory

Begin by checking the shared-memory reference program:

```
/srv/software2/lab/bin/hello_shared_world
```

It uses OpenMP, and the number of threads is controlled through `OMP_NUM_THREADS`. For example:

```
export OMP_NUM_THREADS=4
/srv/software2/lab/bin/hello_shared_world
```

Write a similar program in C, or a script in Python. Each thread should print one line containing at least its thread number, the total number of threads, and the compute-node name. Test the program with different values of `OMP_NUM_THREADS` and verify that it prints exactly one line per thread. The order of the lines is not part of the result.

Hints:

- In C, use OpenMP (preferably) or the `pthread` library. More information can be found in reference guides and tutorials.
- Python's `threading` module can reproduce the thread hierarchy, although ordinary Python code does not necessarily execute concurrently on multiple cores. The worked exercise discusses this limitation in more detail.

Distributed memory

Now check the hybrid MPI and OpenMP reference program:

```
/srv/software2/lab/bin/hello_parallel_world
```

For example, launch four MPI processes with:

```
mpiexec -n 4 /srv/software2/lab/bin/hello_parallel_world
```

Extend your program so that every MPI process creates the requested number of OpenMP threads. Each thread should print one line containing the MPI rank and number of ranks, the thread number and number of threads in that

process, and the compute-node name. Also report the CPU-affinity mask if you reproduce that part of the reference output.

On WILSON, prepare the hierarchical module environment before compiling an MPI program:

```
module use /srv/software/easybuild/hmns/x86 -64 -v2/modules/all/Core
module load GCC/13.2.0
module load OpenMPI/4.1.6
```

Validate the hybrid program with one and several ranks and with different values of `OMP_NUM_THREADS`. For P MPI ranks and T threads per rank, the program should print $P \times T$ lines. Check that every rank is present, that each rank contains thread indices from zero to $T-1$, and that changing `OMP_NUM_THREADS` changes the number of threads rather than the number of MPI processes.

Hints:

- You can get the compute node name with `MPI_Get_processor_name`. Or with `gethostname` defined in `<unistd.h>`, or `socket.gethostname` in Python.
- To get a process/thread CPU affinity mask, you can use `sched_getaffinity` defined in `<sched.h>`, or `os.sched_getaffinity` in Python.

Running on compute nodes

The tests above run on the login node, which is suitable only for very short checks. Real calculations run on the compute nodes and are submitted through Slurm, as described in Batch jobs. Write batch scripts for three kinds of run:

- an OpenMP run: one task with several CPUs per task, setting `OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK`
- an MPI run: several tasks with one CPU each and `OMP_NUM_THREADS=1`;
- a hybrid run: several tasks with several CPUs each, on two nodes.

Launch the program with `srun` inside the batch script, and load in the script the same modules used to compile it. For each run, check that the number of output lines, the ranks, the thread numbers, and the node names agree with the Slurm request.

Report checklist

- the shared-memory program and its output for different values of `OMP_NUM_THREADS`, showing one line per thread;
- the hybrid MPI and OpenMP program and the commands used to build and launch it;

- the validation runs with one and several ranks and different thread counts, including the check of the $P \times T$ lines, the ranks, and the thread indices;
- the batch scripts and output of the OpenMP, MPI, and hybrid runs on the compute nodes;
- the CPU-affinity masks, if you reproduce that part of the reference output.